Step 2 — The Three Extractor Agents — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds three separate specialist chains — one for action items, one for decisions, one for the summary — each a small, focused LCEL chain reading the same transcript. **If not:** one giant "give me everything" prompt does each individual part worse than a focused chain would, and you couldn't test or swap out one piece without touching all the rest.

Each agent is a prompt plus a model call. Two use `with_structured_output` (typed lists); the summarizer uses `StrOutputParser` (plain text). Every chain takes the same input shape — `{"transcript": ...}` — so Step 3 can run them side by side.

The action-item extractor

```
# src/agents/actions.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import ActionItems

ACTIONS_PROMPT = (
    "From the meeting transcript below, list the action items as tasks with "
    "an owner. If an item has no clear owner, leave the owner empty. If there "
    "are none, return an empty list.\n\nTranscript:\n{transcript}"
)


def build_actions_chain(model):
    prompt = ChatPromptTemplate.from_template(ACTIONS_PROMPT)
    return prompt | model.with_structured_output(ActionItems)
```

The decisions extractor

```
# src/agents/decisions.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import Decisions
```

```
DECISIONS_PROMPT = (
    "From the meeting transcript below, list the decisions that were made, "
    "one short sentence each. If none were made, return an empty list.\n\n"
    "Transcript:\n{transcript}"
)


def build_decisions_chain(model):
    prompt = ChatPromptTemplate.from_template(DECISIONS_PROMPT)
    return prompt | model.with_structured_output(Decisions)
```

The summarizer

```
# src/agents/summary.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

SUMMARY_PROMPT = (
    "Summarize the meeting transcript below in 2-3 plain sentences - what "
    "it was about and what came out of it.\n\nTranscript:\n{transcript}"
)


def build_summary_chain(model):
    prompt = ChatPromptTemplate.from_template(SUMMARY_PROMPT)
    return prompt | model | StrOutputParser()
```

Why three chains, not one prompt

- **Each prompt is short and single-purpose.** The action prompt only thinks about actions; the decision prompt only about decisions. A model does one narrow job far more reliably than "extract decisions, actions, and a summary, as JSON."
- **Two typed, one text.** Decisions and actions need a *shape* your code will loop over, so they use `with_structured_output`. The summary is just prose for a human, so `StrOutputParser` is right — don't force a schema on free text.
- **Same input, independent work.** Each chain takes `{"transcript": ...}` and needs nothing from the others — which is exactly what lets Step 3 run all three at the same time.

Each `build_*_chain(model)` returns a runnable; you can `.invoke({"transcript": "..."})` on any of them on its own. The full live run through all three at once is Step 3.